

Embedding and Referencing

Embedding:

- Embedding in MongoDB means you keep related information together in same document.
- This is best suited to the scenario where you have One-To-One relationship or One-To-Many relationship between the entities.
- Embedding means we have denormalized the database, which makes it faster.
- Querying becomes very easy as related information can be fetched together.
- Lesser or no joins always gives better performance.
- Examples:

Scenario 1: E-commerce Product Reviews

- ✓ **Context:** An online shopping platform displays products with customer reviews. Reviews are short, tightly coupled with the product, and frequently fetched alongside the product details.
- ✓ **Use of Embedding:** Storing reviews directly inside the product document prevents separate database lookups, optimizing read performance for the product page.
- ✓ **Syntax:**

```
db.products.insertOne({
  product_id: "prod_9921",
  name: "Wireless Headphones",
  price: 89.99,
  reviews: [
    {
      review_id: "rev_01",
      user: "Pooja",
      rating: 5,
      comment: "Great sound quality and battery life!"
    },
    {
      review_id: "rev_02",
      user: "Sam",
      rating: 4,
      comment: "Comfortable, but the case feels a bit cheap."
    }
  ]
});
```

Scenario 2: User Shipping Addresses

- ✓ **Context:** A food delivery app manages user profiles. Each user can save up to three delivery addresses. These addresses belong exclusively to that user and are updated or deleted only by them.
- ✓ **Use of Embedding:** Treating addresses as an embedded array ensures all location data is captured in a single atomic write, eliminating the need to join a separate addresses collection.
- ✓ **Syntax:**

```
db.users.insertOne({
  user_id: "user_4402",
  username: "Nithya_2004",
  email: "nithya@gmail.com",
  saved_addresses: [
    {
      label: "Home",
      street: "123 Maple St",
      city: "TamilNadu",
      zip: "78701"
    },
    {
      label: "Office",
      street: "500 Sattva Knowledge City",
      city: "Hyderabad",
      zip: "78705"
    }
  ]
});
```

Referencing:

- In this approach data is stored in separate collections and are referenced via their ids.
- This is best suited when information can be created, updated, and accessed independently without affecting each other.
- Referencing means we have normalized the database, which makes the read operations slow.
- Updating the data is independent which makes it easier to handle the information.
- Joins will be required (\$lookup) to fetch the reference.
- Examples:

Scenario 1: E-Commerce Order History

- ✓ **Context:** A customer places multiple orders over time. Each order contains details like a shipping tracking number, total price and billing data.
- ✓ **Use of Referencing:** Storing all orders inside a single user document would eventually break MongoDB's 16MB document size limit if the user is a frequent shopper. Instead, orders are stored in their own collection, and the user document contains an array of references pointing to those orders.
- ✓ **Syntax:**

```
// "users" collection document
{
  "_id": ObjectId("60d5ec49f30b2c3a44111111"),
  "name": "Jane Doe",
  "email": "jane@example.com",
  "order_ids": [
    ObjectId("60d5ec49f30b2c3a44222222"),
    ObjectId("60d5ec49f30b2c3a44333333")
  ]
}
// "orders" collection document
{
  "_id": ObjectId("60d5ec49f30b2c3a44222222"),
  "total_amount": 129.99,
  "status": "Shipped",
  "ship_date": "2026-10-05"
}
```

Scenario 2: Corporate Employee Hierarchy

- ✓ **Context:** A company organization chart tracks who reports to whom across various departments.
- ✓ **Use of Referencing:** An employee can report to a manager, who reports to a director. This is a parent-child relationship within the exact same dataset. Embedding data here is impossible because it creates an infinite loop. Instead, a self-referencing pattern is used where each employee document stores the ID of their manager.
- ✓ **Syntax:**

```
// "employees" collection document (Manager)
{
  "_id": ObjectId("60d5ec49f30b2c3a44cccccc"),
  "name": "Sarah Connor",
  "role": "Engineering Manager",
  "reports_to": null
}
```

```
// "employees" collection document (Direct Report)
{
  "_id": ObjectId("60d5ec49f30b2c3a44ddddd"),
  "name": "John Doe",
  "role": "Software Engineer",
  "reports_to": ObjectId("60d5ec49f30b2c3a44cccc")
}
```

Core Differences:

Feature	Embedding (Denormalized)	Referencing (Normalized)
Structure	Nested sub-documents or arrays inside a parent.	Separate collections linked via Object_Id.
Read Performance	Fast and fetches data in a single query.	Slower and requires multiple queries or \$lookup joins.
Write/Update	Atomic updates for the entire block in one go.	Requires separate updates across multiple collections.
Data Size Limit	Subject to MongoDB's 16 MB document size limit.	No strict limit and collections grow independently.
Best Used For	One-to-one or small one-to-many relationships.	Many-to-many or unbounded growing data.